

# Computer Science (Degree)

## CTSE 2025 - Descriptive Paper I - Questions with Answers

- Maximum marks: 100      Duration: 180 minutes
- Booklet 2537\_25 (I), Series A
- Section A: answer any 8 of Q1-Q10 ( $8 \times 5 = 40$  marks)
- Section B: answer any 3 of Q11-Q15 ( $3 \times 10 = 30$  marks)
- Section C: answer any 2 of Q16-Q19 ( $2 \times 15 = 30$  marks)

### How to use this document

- Every question is followed by a red spoiler bar and a page break. The answer starts on the next page, so you cannot see it while reading the question.
- Answer length is scaled to the marks: roughly 5 marks = 5-6 points, 10 marks = a worked explanation, 15 marks = full treatment with derivation or diagram description.
- Where a diagram is required the document describes exactly what to draw, since a hand-drawn diagram is what the examiner expects.
- For underlying theory that is not specific to one question, see the companion document: CTSE 2025 - Concepts and Revision Notes.docx

## Section A - Q1 to Q10 - 5 marks each (answer any 8)

### Q1 (5 marks)

What are the basic functional units of a computer system? Briefly explain the role of each.

▼ ANSWER BELOW ▼

*Stop here if you are still attempting this question.*

## Answer to Q1

A digital computer is built from five basic functional units, an organisation first described by von Neumann. The arithmetic-logic unit and the control unit together form the CPU.

- Input unit - accepts data and instructions from the outside world and converts them into the binary form the machine works with. Keyboard, mouse, scanner, sensor.
- Memory unit - stores programs and data. Primary memory (RAM, cache) is fast, volatile and directly addressable by the CPU; secondary memory (disk, SSD) is slower, non-volatile and holds data permanently.
- Arithmetic and Logic Unit (ALU) - performs all arithmetic operations (add, subtract, multiply, divide) and all logic operations (AND, OR, NOT, comparison, shifts). Operands are fetched into registers, the operation is performed, and the result goes back to a register or to memory.
- Control unit (CU) - fetches each instruction, decodes it, and generates the timing and control signals that make the other units act in the right order. It does not process data itself; it directs. CU + ALU + registers = CPU.
- Output unit - converts results from internal binary form into a form a human or another machine can use. Monitor, printer, speaker, actuator.

### Key point to add for full marks

The units are connected by the system bus, which carries three kinds of signal: the address bus (which location), the data bus (the value itself) and the control bus (what operation). In the stored-program concept both instructions and data live in the same memory, and that is what makes a computer general purpose rather than fixed function.

**Q2** (5 marks)

Differentiate between an algorithm and a flowchart.

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q2

Both describe the solution to a problem, but at different levels and in different notations. An algorithm is the logic; a flowchart is one way of picturing that logic.

### Algorithm

- A finite, ordered sequence of unambiguous steps that transforms an input into the required output.
- Written in text or pseudocode; language independent.
- Must satisfy finiteness, definiteness, valid input, valid output and effectiveness.
- Compact for long or complex logic, and easy to analyse for time and space complexity.

### Flowchart

- A graphical representation of an algorithm using standard symbols: oval for start and stop, parallelogram for input and output, rectangle for a process, diamond for a decision, arrows for flow of control.
- Easier for a beginner or a non-programmer to follow, and good for showing branching visually.
- Becomes large and unmanageable for big programs, and is hard to modify once drawn.

### Summary of the difference

- Nature: an algorithm is textual and procedural, a flowchart is pictorial.
- Relationship: the algorithm comes first and the flowchart is drawn from it - one algorithm, many possible flowcharts.
- Purpose: an algorithm is for reasoning about and analysing the solution, a flowchart is for communicating and documenting it.
- Both are design tools that precede coding, and neither is executable.

**Q3** (5 marks)

Explain the concept of pointers in C/C++. How are they declared and initialized?

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q3

A pointer is a variable whose value is the memory address of another object. Because it stores an address rather than a value, it lets a program refer to data indirectly.

### Declaration

The syntax is: `data_type *pointer_name;` The type tells the compiler what the pointer points to, which fixes how many bytes a dereference reads and how pointer arithmetic scales.

```
int *p;      /* pointer to int */
float *q;    /* pointer to float */
char *s;     /* pointer to char */
```

### Initialization

A pointer is initialised with the address of an existing object using the address-of operator `&`, or with `NULL` / `nullptr` if it points nowhere yet.

```
int x = 25;
int *p = &x;      /* p now holds the address of x */

int *q;
q = &x;          /* assignment after declaration */

int *r = NULL;    /* C */
int *r2 = nullptr; /* C++ */
```

### Using a pointer

- The dereference operator `*` reads or writes the pointed-to object: `printf("%d", *p)` prints 25, and `*p = 30` changes x to 30.
- Pointer arithmetic is scaled by the size of the pointed-to type, so `p+1` advances by `sizeof(int)` bytes, not by one byte.
- An array name decays to a pointer to its first element, so `arr[i]` is exactly `*(arr + i)`.

### Why pointers matter

- Call by reference: passing an address lets a function modify the caller's variable.
- Dynamic memory: `malloc` and `free` in C, `new` and `delete` in C++, return and take addresses.
- Dynamic data structures: linked lists, trees and graphs are built from nodes that point to each other.
- Efficiency: passing a pointer to a large structure avoids copying it.

*Danger: dereferencing an uninitialised, NULL or dangling pointer is undefined behaviour and typically crashes the program. Always initialise, and set a pointer to NULL after freeing it.*

**Q4** (5 marks)

What are the key differences between a stack and a queue data structure?

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q4

Both are linear data structures that restrict where insertion and deletion may happen. The restriction is the difference.

### Stack

- Discipline: LIFO - last in, first out.
- Access point: one end only, called the top.
- Operations: push (insert), pop (remove), peek or top (inspect), isEmpty, isFull.
- Pointers needed: one (top).
- Applications: function-call and recursion management, expression evaluation and conversion to postfix, undo, backtracking, depth-first search, parenthesis matching.

### Queue

- Discipline: FIFO - first in, first out.
- Access points: two - insert at the rear, remove from the front.
- Operations: enqueue (insert at rear), dequeue (remove from front), front, isEmpty, isFull.
- Pointers needed: two (front and rear).
- Applications: CPU and disk scheduling, print spooling, breadth-first search, buffering, any first-come-first-served resource.

### Common ground

Both can be implemented with an array (fixed capacity, simple) or with a linked list (grows dynamically). In both, push and pop, enqueue and dequeue, are  $O(1)$ . Variants worth naming: the circular queue, which reuses freed slots at the front of an array; the deque, which allows insertion and deletion at both ends; and the priority queue, where removal order is by priority rather than by arrival.

**Q5** (5 marks)

Explain the concept of a linked list. What are its advantages over an array?

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q5

A linked list is a linear collection of nodes in which each node holds a data field and one or more pointers to other nodes. The nodes need not be contiguous in memory; the pointers supply the ordering. A head pointer identifies the first node, and the last node's pointer is NULL.

```
struct Node {  
    int      data;  
    struct Node *next;  
};
```

### Types

- Singly linked - each node points to the next only, so traversal is forward.
- Doubly linked - each node has next and prev, allowing traversal in both directions and easier deletion of a given node.
- Circular - the last node points back to the first, so traversal can start anywhere.

### Advantages over an array

- Dynamic size: nodes are allocated as needed, so there is no fixed capacity decided in advance and no wasted reserved space.
- Cheap insertion and deletion: once you hold a pointer to the position, insertion or deletion is  $O(1)$  pointer rewiring. In an array the same operation is  $O(n)$ , because every following element must shift.
- No contiguous block required: a linked list can use scattered free memory, so it succeeds where an array of the same total size would fail because of fragmentation.
- A natural base for other structures: stacks, queues, trees, graphs, hash-table chains and adjacency lists are all built from linked nodes.

### The cost of those advantages

- No random access: reaching the  $i$ -th element is  $O(n)$  sequential traversal, against  $O(1)$  for an array.
- Extra memory for the pointer in every node.
- Poor cache locality, because the nodes are scattered, so a real-world traversal is slower than an array scan even at the same asymptotic cost.

**Q6** (5 marks)

Define the Big-Oh notation. What does it represent in the context of algorithm analysis?

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q6

### Definition

For functions  $f(n)$  and  $g(n)$  mapping positive integers to non-negative reals,  $f(n) = O(g(n))$  if there exist positive constants  $c$  and  $n_0$  such that:

$$f(n) \leq c * g(n) \quad \text{for all } n \geq n_0$$

In words: beyond some input size  $n_0$ ,  $f(n)$  never grows faster than a constant multiple of  $g(n)$ . Big-Oh is therefore an asymptotic upper bound.

### What it represents in algorithm analysis

- Growth rate, not running time. It says how the cost scales as the input grows, not how many seconds the program takes.
- A guarantee.  $O(g(n))$  is normally quoted for the worst case, so it is an assurance that the algorithm will not do worse than that.
- Machine independence. Constants and lower-order terms are dropped, so the measure does not depend on processor speed, language or compiler:  $3n^2 + 5n + 100$  is simply  $O(n^2)$ .
- A basis for comparison. Two algorithms for the same problem can be ranked without implementing either.

### The usual hierarchy

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

### Related notations

- $\Omega(g(n))$  - asymptotic lower bound,  $f(n) \geq c * g(n)$  for large  $n$ ; the best case.
- $\Theta(g(n))$  - tight bound, both upper and lower:  $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ .

*Because Big-Oh is only an upper bound, saying binary search is  $O(n^2)$  is technically true but useless. Always quote the tightest bound you can, which for binary search is  $O(\log n)$ .*

**Q7** (5 marks)

What is Boolean algebra? Mention its basic operators and their functionalities.

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q7

Boolean algebra, formulated by George Boole, is the algebra of variables that can take only two values, 0 and 1 (false and true, low and high). It is the mathematical foundation of digital logic design: every combinational circuit is a Boolean function, and Boolean identities are the tool used to simplify a circuit into fewer gates.

### The three basic operators

- AND (logical product), written  $A \cdot B$  or  $AB$ . The output is 1 only when every input is 1. Truth table: 00→0, 01→0, 10→0, 11→1.
- OR (logical sum), written  $A + B$ . The output is 1 when at least one input is 1. Truth table: 00→0, 01→1, 10→1, 11→1.
- NOT (complement or inversion), written  $A'$  or  $\bar{A}$ . Single input; the output is the opposite of the input. 0→1, 1→0.

### Derived operators

- NAND =  $(AB)'$  and NOR =  $(A+B)'$ . Each is universal - any Boolean function can be built from NAND gates alone, or from NOR gates alone - which is why they dominate real IC design.
- XOR ( $A \text{ xor } B$ ) outputs 1 when the inputs differ; used in adders, parity generators and comparators. XNOR outputs 1 when they are equal.

### Important laws

- Identity:  $A + 0 = A$ ,  $A \cdot 1 = A$ . Null:  $A + 1 = 1$ ,  $A \cdot 0 = 0$ .
- Idempotent:  $A + A = A$ ,  $A \cdot A = A$ . Complement:  $A + A' = 1$ ,  $A \cdot A' = 0$ .
- Commutative, associative and distributive:  $A(B + C) = AB + AC$ .
- Absorption:  $A + AB = A$ . Involution:  $(A')' = A$ .
- De Morgan's theorems:  $(A + B)' = A'B'$  and  $(AB)' = A' + B'$ .

**Q8** (5 marks)

Differentiate between combinational and sequential circuits. Give one example of each.

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q8

### Combinational circuits

- The output depends only on the present combination of inputs.
- No memory and no feedback path, so the circuit has no notion of history or state.
- No clock is required; the output changes as soon as the inputs change, after the propagation delay.
- Fully described by a truth table or a Boolean expression.
- Designed with gates and simplified using Boolean algebra or Karnaugh maps.
- Example: a 4-bit binary adder. Others: multiplexer, decoder, encoder, comparator, code converter.

### Sequential circuits

- The output depends on the present inputs and on the past sequence of inputs, that is on the stored state.
- Contain memory elements (flip-flops or latches) and feedback from output back to input.
- Normally clocked, so the state changes at defined instants (synchronous). Asynchronous sequential circuits change as soon as the inputs change.
- Described by a state table, a state diagram and excitation equations, not by a truth table alone.
- Example: a 4-bit up counter. Others: shift register, flip-flop, finite-state machine, RAM control.

### One-line distinction

A combinational circuit computes a function; a sequential circuit computes a function and remembers. Every practical digital system is a mixture: combinational logic computes the next state, and flip-flops hold it.

**Q9** (5 marks)

Explain the concept of addressing modes in computer architecture. Give one example.

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q9

An addressing mode is the rule the CPU uses to work out the effective address of an instruction's operand - that is, how the address field of the instruction is to be interpreted. Several modes exist so that the instruction set can support constants, variables, arrays, pointers, records and position-independent code without wasting instruction bits.

### The common modes

- Immediate - the operand itself is in the instruction, so there is no memory access. Example: `MOV R1, #5` loads the constant 5.
- Register - the operand is in a named register. Fastest, no memory access. Example: `ADD R1, R2`.
- Direct (absolute) - the instruction holds the memory address of the operand. Example: `LOAD R1, 2000` loads the contents of location 2000.
- Indirect - the instruction holds an address that contains the address of the operand, so there are two memory accesses. Example: `LOAD R1, (2000)`.
- Register indirect - a register holds the operand's address. This is how pointers are implemented. Example: `LOAD R1, (R2)`.
- Indexed - effective address = base address + contents of an index register. Made for arrays: the base is the start of the array and the index register holds the subscript.
- Base plus displacement - effective address = contents of a base register + a constant offset held in the instruction. Used for record fields and stack frames.
- Relative - effective address = program counter + offset. Used for branches and for position-independent code.
- Implied (implicit) - the operand is fixed by the opcode itself, so no address field is needed. Example: `CLC` clears the carry flag; `PUSH` and `POP` imply the stack pointer.

### Worked example of indexed mode

To fetch `A[6]` where array `A` begins at 2000: put 6 in index register `X` and execute `LOAD R1, 2000(X)`. The CPU computes the effective address  $2000 + 6 = 2006$  and loads the contents of 2006. Changing only `X` walks the whole array with one instruction, which is why indexed mode makes loops over arrays efficient.

**Q10** (5 marks)

What is the role of a linker in the compilation process?

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q10

The linker is the tool that runs after the compiler or assembler and before the loader. It takes one or more relocatable object modules and combines them into a single executable image. Its existence is what makes separate compilation possible.

### Its main jobs

- Combining modules - merges the corresponding sections (text or code, data, bss, read-only data) of all input object files into unified sections in the output.
- Symbol resolution - each object file exports the symbols it defines and lists the external symbols it uses but does not define. The linker matches every external reference to exactly one definition. Failing to find one produces the familiar undefined reference error; finding two produces a multiple definition error.
- Relocation - the compiler emits each module as if it started at address 0. Once the linker decides the final layout, it patches every address-dependent instruction and data word to the correct final address, using the relocation records the compiler left behind.
- Library linking - searches the static libraries named on the command line and pulls in only those object modules that satisfy still-unresolved references.
- Producing the output - writes the executable with its header, section table and entry point and, for dynamic executables, the list of shared libraries needed at run time.

### Static versus dynamic linking

- Static linking copies library code into the executable at link time. The result is a larger, self-contained binary, and a library bug fix needs a relink.
- Dynamic linking records only a reference; the shared library (.dll, .so) is bound at load time or on first use by the dynamic linker. The result is smaller binaries, one copy of the library shared in memory, and library updates without relinking - at the cost of a run-time dependency.

### Where it sits

Source -> preprocessor -> compiler -> assembler -> object file -> LINKER -> executable -> loader -> running process. Note the difference from the loader: the linker resolves references between modules at build time, while the loader brings the finished image into memory and starts it.

## Section B - Q11 to Q15 - 10 marks each (answer any 3)

### Q11 (10 marks)

Write a C/C++ function to reverse an array. Explain the logic and provide a simple example of its usage.

▼ ANSWER BELOW ▼

*Stop here if you are still attempting this question.*

## Answer to Q11

### The logic

Reverse the array in place using two indices that walk towards each other. Put left at the first element and right at the last, swap the two elements, then move left forward and right backward. Stop when they meet or cross. Each iteration fixes two elements, so the loop runs  $n/2$  times.

- Time complexity:  $O(n)$  - each element is touched once.
- Space complexity:  $O(1)$  - one temporary variable, no second array.
- The loop condition  $\text{left} < \text{right}$  is what makes an odd-length array work: the middle element is already in its correct place and is left alone.

### The function

```
#include <stdio.h>

void reverseArray(int arr[], int n)
{
    int left  = 0;
    int right = n - 1;
    int temp;

    while (left < right) {
        temp      = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;

        left++;
        right--;
    }
}
```

### Example of usage

```
int main(void)
{
    int a[] = {10, 20, 30, 40, 50};
    int n = sizeof(a) / sizeof(a[0]);
    int i;

    printf("Before: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);

    reverseArray(a, n);

    printf("\nAfter : ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);

    return 0;
}
```

## Output

Before: 10 20 30 40 50

After : 50 40 30 20 10

## Dry run

- Start {10,20,30,40,50}, left=0, right=4. Swap 10 and 50 -> {50,20,30,40,10}.
- left=1, right=3. Swap 20 and 40 -> {50,40,30,20,10}.
- left=2, right=2. The condition left < right fails, the loop ends, and 30 stays in the middle.

## Points worth adding

- The array is passed by reference automatically: an array name decays to a pointer, so the function modifies the caller's array and needs no return value. The length must be passed separately because that information is lost in the decay.
- Recursive alternative: swap the ends, then recurse on the inner part with reverse(arr, left+1, right-1) and base case left >= right. Same O(n) time but O(n) stack space.
- In C++ the same result is one line: std::reverse(a, a + n). In an examination or interview, write the loop unless told otherwise.
- Defensive check: if (arr == NULL || n <= 1) return; handles the empty, single-element and null cases.

**Q12** (10 marks)

Explain the working principle of the Quick Sort algorithm. What is its average and worst-case time complexity?

▼ ANSWER BELOW ▼

*Stop here if you are still attempting this question.*

## Answer to Q12

### Working principle

Quick Sort is a divide-and-conquer, in-place comparison sort. The work is done in the divide step rather than in the combine step, which is the opposite of Merge Sort.

1. Choose a pivot element from the array - first, last, middle, or a random or median-of-three choice.
2. Partition: rearrange the array so that every element less than the pivot lies to its left and every element greater lies to its right. The pivot is now in its final sorted position.
3. Recur on the sub-array to the left of the pivot and on the sub-array to the right.
4. Base case: a sub-array of length 0 or 1 is already sorted, so the recursion stops.

No merge step is needed: once every pivot has been placed correctly and both sides sorted, the whole array is sorted.

### Lomuto partition scheme

```
int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1, j, t;

    for (j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            t = a[i]; a[i] = a[j]; a[j] = t;
        }
    }
    i++;
    t = a[i]; a[i] = a[high]; a[high] = t;
    return i;          /* final pivot index */
}

void quickSort(int a[], int low, int high)
{
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

### Complexity

- Best case  $O(n \log n)$ . The pivot splits the array into two nearly equal halves every time, so the recursion depth is  $\log n$  and each level does  $O(n)$  partitioning work.
- Average case  $O(n \log n)$ . Averaged over all input permutations the splits are good enough, and the constant factor is small, which is why Quick Sort is usually the fastest general-purpose sort in practice.
- Worst case  $O(n^2)$ . The pivot is the smallest or the largest element every time, so one side is empty and the other holds  $n-1$  elements. The recursion depth becomes  $n$  and the total work is  $n + (n-1) + (n-2) + \dots$

=  $O(n^2)$ . With a first-element or last-element pivot this happens on already sorted or reverse-sorted input, which are exactly the cases you might expect to be easy.

### **Space complexity**

- $O(\log n)$  auxiliary on average for the recursion stack, and  $O(n)$  in the worst case. It sorts in place, so no second array is needed.

### **Other properties, and how to avoid the worst case**

- Not stable - equal elements can be reordered by the swaps.
- Randomised pivot selection makes the  $O(n^2)$  case vanishingly unlikely for any fixed input.
- Median-of-three (first, middle, last) is a cheap deterministic improvement.
- Switching to insertion sort for small sub-arrays, about ten elements, reduces recursion overhead.
- Introsort, used in the C++ standard library, starts with Quick Sort and switches to Heap Sort once the recursion becomes too deep, giving a guaranteed  $O(n \log n)$ .

### **Comparison with Merge Sort**

Merge Sort guarantees  $O(n \log n)$  and is stable, but needs  $O(n)$  extra space. Quick Sort risks  $O(n^2)$  and is unstable, but sorts in place with better cache behaviour and a smaller constant. Quick Sort is preferred for arrays in memory, Merge Sort for linked lists and for external sorting.

**Q13** (10 marks)

Describe the different levels of memory organization in a computer system. Explain the purpose of the memory hierarchy.

▼ ANSWER BELOW ▼

*Stop here if you are still attempting this question.*

## Answer to Q13

No single memory technology is simultaneously fast, large and cheap. Memory is therefore organised as a hierarchy of levels: as you move away from the CPU, speed and cost per bit fall while capacity rises.

### The levels, fastest first

- CPU registers - a few hundred bytes, access in one clock cycle. They hold the operands the ALU is working on right now. Fastest and most expensive per bit.
- Cache memory (SRAM) - L1 typically 32 to 64 KB per core with access in 1 to 4 cycles; L2 a few hundred KB to a few MB; L3 several MB shared between cores, tens of cycles. Holds recently and frequently used instructions and data.
- Main memory (DRAM) - gigabytes, access of the order of 50 to 100 ns or a few hundred cycles. Holds the programs and data of all running processes. Volatile.
- Secondary storage (SSD, hard disk) - hundreds of gigabytes to terabytes. SSD access in tens of microseconds, magnetic disk in milliseconds. Non-volatile; holds the file system and backs virtual memory through paging and swapping.
- Tertiary or offline storage - magnetic tape, optical disc, cloud archive. Very large and very cheap per bit, access in seconds to minutes. Used for backup and archival.

### Purpose of the hierarchy

- To give the illusion of a memory that is as large as the slowest level and almost as fast as the fastest level, at a cost close to that of the cheapest level.
- To close the processor-memory performance gap. CPU speed has grown far faster than DRAM speed; without cache the CPU would stall for most of its cycles waiting on memory.
- To exploit locality of reference, which is what makes the whole scheme work. Temporal locality: an item used now is likely to be used again soon, so keep it in a fast level. Spatial locality: items near a used address are likely to be used soon, so fetch a whole block rather than a single word.

### How the levels cooperate

- Data moves between adjacent levels in fixed-size units - cache lines or blocks between cache and main memory, pages between main memory and disk.
- On a hit the item is found at that level. On a miss it is fetched from the level below and a resident item is evicted according to a replacement policy such as LRU.
- Average access time = hit time + miss rate x miss penalty. A high hit rate matters more than the raw speed of a level, which is why cache design targets hit rate.
- Write policies: write-through updates both levels immediately and keeps them consistent; write-back updates only the cache and writes to memory on eviction, reducing traffic at the cost of a dirty bit and extra consistency management.
- The hierarchy is largely transparent: the cache is managed entirely by hardware, and virtual memory is managed by the operating system with the MMU, so the programmer sees one flat address space.

*Worth stating explicitly: registers and cache are managed by hardware, main memory and swap by the operating system, and archival storage by the user or administrator.*

**Q14** (10 marks)

Design a combinational circuit for a 2-bit binary adder. Draw the logic diagram and explain its operation.

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q14

A 2-bit binary adder adds two 2-bit numbers  $A = A_1A_0$  and  $B = B_1B_0$  and produces a 2-bit sum  $S_1S_0$  together with a carry-out  $C_2$ . It is built from one half adder and one full adder, or from two full adders with the first carry-in tied to 0.

### Step 1 - the half adder

Adds two bits and produces a sum and a carry.

$$\begin{array}{ll} \text{Sum } S &= A \text{ xor } B \quad (\text{XOR gate}) \\ \text{Carry } C &= A \cdot B \quad (\text{AND gate}) \end{array}$$

| A | B |  | S | C |
|---|---|--|---|---|
| 0 | 0 |  | 0 | 0 |
| 0 | 1 |  | 1 | 0 |
| 1 | 0 |  | 1 | 0 |
| 1 | 1 |  | 0 | 1 |

### Step 2 - the full adder

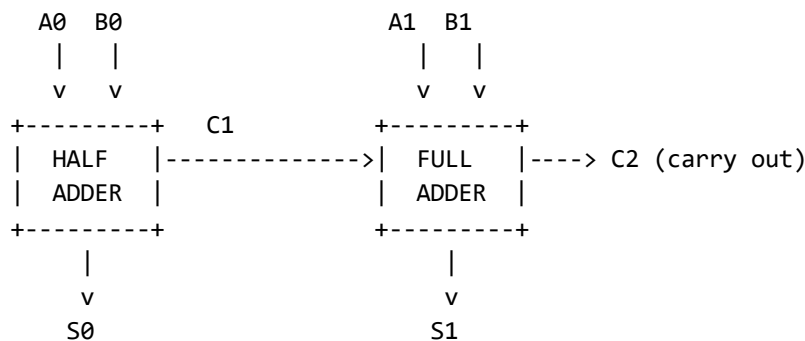
Adds three bits: two operand bits and a carry-in.

$$\begin{array}{ll} \text{Sum } S &= A \text{ xor } B \text{ xor } C_{in} \\ \text{Carry } C_{out} &= A \cdot B + C_{in} \cdot (A \text{ xor } B) \end{array}$$

| A | B | Cin |  | S | Cout |
|---|---|-----|--|---|------|
| 0 | 0 | 0   |  | 0 | 0    |
| 0 | 0 | 1   |  | 1 | 0    |
| 0 | 1 | 0   |  | 1 | 0    |
| 0 | 1 | 1   |  | 0 | 1    |
| 1 | 0 | 0   |  | 1 | 0    |
| 1 | 0 | 1   |  | 0 | 1    |
| 1 | 1 | 0   |  | 0 | 1    |
| 1 | 1 | 1   |  | 1 | 1    |

A full adder is two half adders plus one OR gate: the first half adder adds A and B, the second adds that sum to  $C_{in}$ , and the OR gate combines the two carries.

### Step 3 - the 2-bit ripple carry adder



### Explanation of the operation

- Stage 0, the least significant stage: there is no incoming carry, so a half adder suffices. It produces  $S_0 = A_0 \text{ xor } B_0$  and  $C_1 = A_0.B_0$ .
- Stage 1: a full adder adds  $A_1$ ,  $B_1$  and the carry  $C_1$  produced by stage 0. It produces  $S_1 = A_1 \text{ xor } B_1 \text{ xor } C_1$  and the final carry-out  $C_2$ .
- The result is the three-bit value  $C_2 S_1 S_0$ , so  $11 + 11 = 110$ , that is  $3 + 3 = 6$ , and the carry-out is needed to represent the answer.

**Worked example:  $A = 11$ ,  $B = 01$**

- Stage 0:  $A_0=1$ ,  $B_0=1$ , so  $S_0 = 1 \text{ xor } 1 = 0$  and  $C_1 = 1$ .
- Stage 1:  $A_1=1$ ,  $B_1=0$ ,  $C_1=1$ , so  $S_1 = 1 \text{ xor } 0 \text{ xor } 1 = 0$  and  $C_2 = 1.0 + 1.(1 \text{ xor } 0) = 0 + 1 = 1$ .
- Result  $C_2 S_1 S_0 = 100 = 4$ , and indeed  $3 + 1 = 4$ .

**Points that earn the last marks**

- This is a ripple carry adder: the carry must propagate through every stage, so the worst-case delay is proportional to the number of bits. For an  $n$ -bit adder with delay  $t$  per stage the total is about  $n.t$ , which limits speed.
- A carry look-ahead adder computes all carries in parallel from generate terms ( $G_i = A_i.B_i$ ) and propagate terms ( $P_i = A_i \text{ xor } B_i$ ), trading more gates for logarithmic delay.
- Subtraction reuses the same hardware: complement  $B$  and set the first carry-in to 1, which gives the two's complement difference  $A - B$ .
- Overflow for signed operands is detected as  $C_2 \text{ xor } C_1$ , that is a carry into the sign position differing from the carry out of it.

**Q15** (10 marks)

Discuss the different frameworks and tools commonly used in mobile application development.

▼ **ANSWER BELOW** ▼

*Stop here if you are still attempting this question.*

## Answer to Q15

Mobile development divides into native development, which targets one platform with its own language and SDK, and cross-platform development, which shares one codebase across platforms. The choice is a trade-off between performance and platform fidelity on one side and development cost and time to market on the other.

### Native frameworks

- Android - Kotlin, official since 2019, or Java, built in Android Studio with the Android SDK, Gradle and the Jetpack libraries. Jetpack Compose is the modern declarative UI toolkit, replacing XML layouts.
- iOS - Swift, or the older Objective-C, built in Xcode with UIKit or the newer declarative SwiftUI, and distributed through the App Store.
- Strengths: best performance, immediate access to every new OS API and device feature, and a look and feel that matches the platform exactly.
- Weakness: two separate codebases and two skill sets, so roughly double the effort.

### Cross-platform frameworks

- Flutter - Google's toolkit using the Dart language. It renders its own widgets with the Skia or Impeller engine rather than using native controls, so the interface is identical everywhere. Compiles ahead of time to native ARM code and has strong hot reload. Targets Android, iOS, web and desktop.
- React Native - Meta's framework using JavaScript or TypeScript with React. Maps components onto real native widgets through a bridge, now the JSI and Fabric architecture. Large ecosystem, and Expo simplifies build and deployment.
- .NET MAUI - Microsoft's evolution of Xamarin.Forms, using C# and XAML over a shared .NET codebase with native controls.
- Kotlin Multiplatform - shares business logic in Kotlin while keeping the user interface native on each platform. A middle path that avoids compromising the interface.
- Ionic with Capacitor - the application is a web application in HTML, CSS and JavaScript, often with Angular or React, running inside a native web view with plugin access to device APIs. Fastest to build, weakest performance for graphics-heavy work.

### Supporting tools

- Backend and services: Firebase (authentication, Firestore, Cloud Functions, Cloud Messaging for push, Crashlytics), AWS Amplify, Supabase, or a custom REST or GraphQL API.
- Local storage: SQLite, Room on Android, Core Data on iOS, Realm, or key-value stores such as SharedPreferences and UserDefaults.
- Design and prototyping: Figma and Adobe XD, plus the platform design systems - Material Design for Android and the Human Interface Guidelines for iOS.
- Testing: JUnit and Espresso on Android, XCTest on iOS, Appium and Detox for cross-platform end-to-end testing, and device farms such as Firebase Test Lab or BrowserStack for real-device coverage.
- Build and release: Gradle, the Xcode build system, Fastlane for automation, GitHub Actions or Bitrise or Codemagic for CI/CD, and the Play Console and App Store Connect for distribution.
- Debugging and monitoring: Android Studio Profiler, Xcode Instruments, Flipper, Sentry or Crashlytics for crash reporting, and an analytics platform for usage data.

## **How to choose**

- Choose native for graphics-intensive games, heavy camera or sensor work, or when new OS features must be adopted immediately.
- Choose Flutter or React Native for standard business and content applications, where one team and one codebase matter more than the last few percent of performance.
- Choose a web-view approach only for simple content-driven applications, or when an existing web application must be shipped to the stores quickly.

## Section C - Q16 to Q19 - 15 marks each (answer any 2)

### Q16 (15 marks)

Explain the design and working of a JK flip-flop. How can it be used to implement other types of flip-flops?

▼ ANSWER BELOW ▼

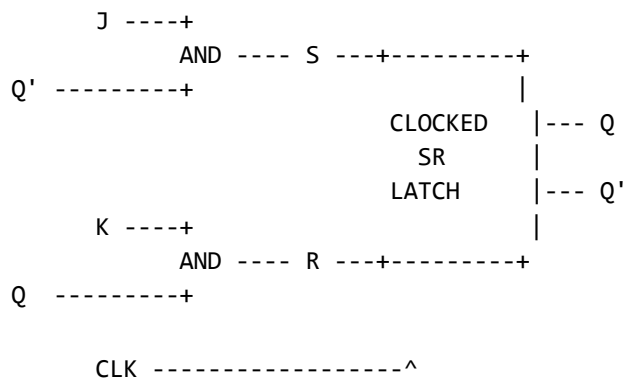
*Stop here if you are still attempting this question.*

## Answer to Q16

The JK flip-flop is the most versatile of the clocked bistable elements. It was designed to remove the one defect of the SR flip-flop: the forbidden input combination  $S = R = 1$ , which drives both outputs to the same level and leaves the next state undefined. The JK flip-flop gives that combination a useful meaning instead - toggle.

### Design

- Start from a clocked SR flip-flop built from two cross-coupled NAND, or NOR, gates with a two-gate input steering network enabled by the clock.
- Add feedback: the J input is ANDed with  $Q'$  and the K input is ANDed with  $Q$ , and these products drive S and R respectively. So  $S = J.Q'$  and  $R = K.Q$ .
- This feedback makes the illegal state unreachable. When  $J = K = 1$ , exactly one of  $Q$  and  $Q'$  is 1, so exactly one of  $S$  and  $R$  is asserted, and the flip-flop is forced to change to the opposite state rather than into an undefined one.



### Truth table, the characteristic table

| J | K | Q(next) | Operation           |
|---|---|---------|---------------------|
| 0 | 0 | Q       | No change (hold)    |
| 0 | 1 | 0       | Reset (clear)       |
| 1 | 0 | 1       | Set                 |
| 1 | 1 | Q'      | Toggle (complement) |

### Characteristic equation

$$Q(\text{next}) = J.Q' + K'.Q$$

This is derived from the truth table with a Karnaugh map on J, K and Q. The single equation captures all four behaviours.

### Excitation table, used when designing counters

| Q | Q(next) | J | K |
|---|---------|---|---|
| 0 | 0       | 0 | X |
| 0 | 1       | 1 | X |
| 1 | 0       | X | 1 |
| 1 | 1       | X | 0 |

The don't-care entries X are the reason JK flip-flops give simpler counter logic than D flip-flops: each transition constrains only one of the two inputs.

### Working, step by step

- $J = 0, K = 0$ :  $S = 0$  and  $R = 0$ , so the latch is undisturbed and the stored bit is held. This is used to make a register ignore a clock edge.
- $J = 0, K = 1$ : if  $Q = 1$  then  $R = 1$  and the flip-flop resets to 0; if  $Q$  is already 0 then  $R = 0$  and it stays 0. The net effect is  $Q(\text{next}) = 0$ .
- $J = 1, K = 0$ : if  $Q = 0$  then  $Q' = 1$  so  $S = 1$  and the flip-flop sets to 1; if  $Q$  is already 1 it stays. The net effect is  $Q(\text{next}) = 1$ .
- $J = 1, K = 1$ : if  $Q = 0$  then  $S = 1$  and it sets; if  $Q = 1$  then  $R = 1$  and it resets. Either way the output complements on every active clock edge. This is the toggle mode that makes counters possible.

### The race-around problem and its cure

- If the flip-flop is level-triggered and the clock pulse width  $t_p$  is greater than the propagation delay  $d_t$ , then with  $J = K = 1$  the output toggles, the new value feeds back, and it toggles again, repeatedly, for as long as the clock is high. The final state is unpredictable. This is race-around.
- Cure 1: make  $t_p < d_t$ . Impractical to guarantee.
- Cure 2, the standard solution: the master-slave configuration. Two JK latches in series, the master enabled on the high half of the clock and the slave on the low half through an inverted clock. The master captures the new value while the slave holds the old one, then the slave copies the master when the clock falls. The output can therefore change only once per clock cycle.
- Cure 3: edge triggering, so the flip-flop responds only to the instant of a rising or falling transition rather than to a level.

### Implementing other flip-flops from a JK

- D flip-flop - connect  $J = D$  and  $K = D'$ .  $J$  and  $K$  are then always complementary, so the toggle and hold rows can never be selected:  $Q(\text{next}) = D \cdot Q' + D' \cdot Q = D$ . One inverter converts a JK into a D.
- T, or toggle, flip-flop - tie  $J$  and  $K$  together and call the common input  $T$ .  $T = 0$  gives  $J = K = 0$ , hold;  $T = 1$  gives  $J = K = 1$ , toggle. The characteristic equation is  $Q(\text{next}) = T \text{ xor } Q$ . This is the building block of ripple counters.
- SR flip-flop - use  $J$  as  $S$  and  $K$  as  $R$  directly. The behaviour matches for the three legal combinations; the difference is that the JK toggles where the SR is undefined, so the JK is a strict improvement.
- JK from a D - the reverse direction also works: drive  $D$  with the characteristic equation  $D = J \cdot Q' + K' \cdot Q$ , using two AND gates, an OR gate and an inverter.

### Applications

- Toggle mode gives frequency division by two, and cascading  $n$  flip-flops divides by  $2^n$ .
- Asynchronous, or ripple, and synchronous binary counters - up, down and up/down.
- Shift registers, including the SISO, SIPO, PISO and PIPO forms.
- Bounce elimination, sequence detectors, and general finite-state-machine state storage.

**Q17** (15 marks)

Describe and compare the divide and conquer and greedy algorithmic paradigms. Provide a concrete example of a problem solved using each approach.

▼ ANSWER BELOW ▼

*Stop here if you are still attempting this question.*

## Answer to Q17

### Divide and conquer

A problem is solved by breaking it into smaller instances of the same problem, solving those independently, and combining their solutions. It has three phases:

5. Divide - split the problem into two or more sub-problems of the same type, usually of roughly equal size.
  6. Conquer - solve each sub-problem recursively; when a sub-problem is small enough, solve it directly. That is the base case.
  7. Combine - merge the sub-solutions into a solution to the original problem.
- The sub-problems must be independent, that is they must not overlap. If they do overlap, the same sub-problem is recomputed exponentially often and the right paradigm is dynamic programming instead.
  - Analysis uses a recurrence such as  $T(n) = a.T(n/b) + f(n)$ , solved with the Master Theorem or a recursion tree.
  - Naturally parallel: independent sub-problems can be solved on different cores.

### The greedy method

A solution is built one piece at a time, and at each step the option that looks best right now is chosen irrevocably. There is no backtracking and no reconsideration.

- It works only when the problem has the greedy-choice property, that a locally optimal choice can be extended to a globally optimal solution, together with optimal substructure.
- When those properties fail, a greedy algorithm still produces a valid answer, but not necessarily the best one. The 0/1 knapsack problem is the standard counter-example: greedy by value density can be beaten.
- Typically much faster than exhaustive search, and often dominated by an initial sort, giving  $O(n \log n)$ .
- Correctness must be proved, usually by an exchange argument or a cut-and-paste argument. It cannot be assumed.

### Example of divide and conquer: Merge Sort

- Divide: split the array at the midpoint into two halves.
- Conquer: sort each half recursively; a single element is already sorted.
- Combine: merge the two sorted halves by repeatedly taking the smaller of the two front elements.
- The recurrence  $T(n) = 2T(n/2) + O(n)$  solves to  $\Theta(n \log n)$  in all cases - best, average and worst.
- It uses  $O(n)$  auxiliary space for the merge, and it is stable.

```
MergeSort(A, low, high):  
    if low < high:  
        mid = (low + high) / 2  
        MergeSort(A, low, mid)  
        MergeSort(A, mid + 1, high)  
        Merge(A, low, mid, high)
```

Trace on {38, 27, 43, 3}: split into {38,27} and {43,3}; split again into singletons; merge into {27,38} and {3,43}; final merge gives {3,27,38,43}.

### **Example of a greedy algorithm: fractional knapsack**

- Problem: a knapsack of capacity  $W$  and  $n$  items, each with a value and a weight. Items may be broken into fractions. Maximise the total value carried.
- Greedy choice: compute value/weight for every item, sort in decreasing order of that ratio, take items whole while they fit, then take a fraction of the next item to fill the remaining capacity exactly.
- Complexity  $O(n \log n)$ , dominated by the sort.
- Worked example,  $W = 50$  with items  $(60, 10)$ ,  $(100, 20)$ ,  $(120, 30)$ . The ratios are 6, 5 and 4. Take item 1 whole for value 60 with 40 left, item 2 whole for value 100 with 20 left, then  $20/30$  of item 3 for value 80. Total 240, which is optimal.
- Why greedy is provably optimal here: because fractions are allowed, any solution that does not fill the sack with the highest available ratios can be improved by swapping in a higher-ratio unit - the exchange argument. Remove the fractional allowance and that argument collapses, which is exactly why 0/1 knapsack needs dynamic programming.

### **Comparison**

- Structure: divide and conquer is recursive and top-down; greedy is iterative and incremental.
- Choices: divide and conquer solves all sub-problems and combines them; greedy makes one choice per step and discards the alternatives.
- Reconsideration: divide and conquer explores whole sub-problems; greedy never revisits a decision.
- Combine step: essential in divide and conquer, the merge; absent in greedy.
- Optimality: divide and conquer gives an exact answer whenever the decomposition is correct; greedy gives an optimal answer only when the greedy-choice property holds.
- Typical cost: divide and conquer is often  $O(n \log n)$  from the recurrence; greedy is often  $O(n \log n)$  from a sort, with a smaller constant.
- Failure mode: divide and conquer degrades badly when sub-problems overlap; greedy silently returns a sub-optimal answer when the greedy-choice property fails.

### **Other examples worth naming**

- Divide and conquer: binary search, Quick Sort, Strassen's matrix multiplication, closest pair of points, Karatsuba multiplication, maximum subarray.
- Greedy: Dijkstra's shortest path, Prim's and Kruskal's minimum spanning trees, Huffman coding, activity selection, job sequencing with deadlines.

**Q18** (15 marks)

Elaborate on the different phases of a compiler, from lexical analysis to code generation. Explain the primary task and output of each phase.

▼ ANSWER BELOW ▼

*Stop here if you are still attempting this question.*

## Answer to Q18

A compiler translates a program in a source language into an equivalent program in a target language, reporting errors as it goes. The work is divided into phases, grouped into the analysis phase or front end, which understands the source and builds an intermediate representation, and the synthesis phase or back end, which produces target code from it.

### Phase 1 - Lexical analysis, or scanning

- Task: read the source character by character, group characters into lexemes and emit a token for each. Strip white space and comments. Enter identifiers and constants into the symbol table. Track line numbers for error messages.
- Implemented as a finite automaton, generated from regular expressions by a tool such as Lex or Flex.
- Output: a stream of tokens of the form <token-name, attribute-value>.
- Errors detected: illegal characters, unterminated strings or comments, malformed numeric literals.

Source:    position = initial + rate \* 60

Tokens:    <id,1> <=> <id,2> <+> <id,3> <\*> <number,60>

### Phase 2 - Syntax analysis, or parsing

- Task: check that the token stream conforms to the grammar of the language and build a parse tree, or syntax tree, that exhibits the grammatical structure. The grammar is a context-free grammar, usually written in BNF.
- Techniques: top-down, that is recursive descent or LL(1) with a predictive table; or bottom-up, that is shift-reduce, LR, SLR or LALR as produced by Yacc or Bison.
- Output: a syntax tree, or an abstract syntax tree in which the punctuation of the grammar has been discarded.
- Errors detected: missing semicolon, unbalanced brackets, malformed expression - anything structurally invalid.

```
      =
     / \
id:1   +
     / \
id:2   *
     / \
id:3  60
```

### Phase 3 - Semantic analysis

- Task: check the meaning-related rules that a context-free grammar cannot express. Verify type compatibility of operands; that every identifier is declared before use and not declared twice in one scope; that function calls match their signatures in number and type of arguments; that array subscripts are integers; and that break and continue appear in legal places. Insert implicit type conversions, or coercions, where the language allows them.
- Relies heavily on the symbol table, and is often expressed as a syntax-directed definition with attributes attached to the tree.
- Output: an annotated syntax tree, decorated with types and other attributes.

- Errors detected: type mismatch, undeclared identifier, wrong number of arguments.

In the running example 60 is an integer while rate is real, so the analyser inserts a widening conversion, `inttofloat(60)`.

#### Phase 4 - Intermediate code generation

- Task: translate the annotated tree into a machine-independent intermediate representation. This decouples the front end from the back end, so  $m$  languages and  $n$  machines need  $m + n$  components rather than  $m \times n$  complete compilers.
- Common forms: three-address code, quadruples, triples, postfix notation, static single assignment form.
- Output: intermediate code, which is easy to translate and easy to optimise.

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

#### Phase 5 - Code optimization

- Task: improve the intermediate code so that the target program runs faster or occupies less space, without altering its observable behaviour. This phase is optional in principle and the largest part of a production compiler in practice.
- Machine-independent techniques: constant folding, constant propagation, common sub-expression elimination, dead-code elimination, copy propagation, strength reduction, loop-invariant code motion, loop unrolling, induction-variable elimination.
- Scope: local, within a basic block; global, across a whole procedure using data-flow analysis on the control-flow graph; or interprocedural.
- Output: optimised intermediate code.

```
t1 = id3 * 60.0
id1 = id2 + t1
```

Here the conversion was folded at compile time and one temporary was eliminated.

#### Phase 6 - Code generation

- Task: map the optimised intermediate code onto the instruction set of the target machine. There are three sub-problems: instruction selection, register allocation and assignment, and instruction ordering or scheduling.
- Register allocation is the hardest part and is usually solved by graph colouring on the interference graph; values that will not fit in registers are spilled to memory.
- Output: target assembly or machine code.

```
LDF  R2, id3
MULF R2, R2, #60.0
LDF  R1, id2
ADDF R1, R1, R2
STF  id1, R1
```

#### Two components that span every phase

- Symbol table - a data structure holding one record per identifier with its name, type, scope, storage class, size and memory offset. It is created by the lexical analyser and consulted or updated by every later phase. Usually implemented as a hash table for  $O(1)$  average lookup.
- Error handler - detects errors, reports them with a useful location and message, and recovers so that compilation can continue and report more than one error per run. Recovery strategies include panic mode, phrase-level correction, error productions and global correction.

#### **Related points for full marks**

- Passes versus phases: several phases may be combined into a single pass over the program. A one-pass compiler is fast; a multi-pass compiler can optimise better.
- The front end, phases 1 to 4, is source-language dependent and machine independent; the back end, phases 5 and 6, is machine dependent. This split is what allows GCC and LLVM to support many languages on many targets.
- Distinguish the compiler from its neighbours: the preprocessor expands macros and includes, the assembler turns assembly into relocatable object code, the linker resolves references between modules, and the loader places the image in memory.

**Q19** (15 marks)

Discuss the challenges and considerations involved in designing the right User Interface (UI) for mobile applications.

▼ ANSWER BELOW ▼

*Stop here if you are still attempting this question.*

## Answer to Q19

Designing a mobile interface is constrained in ways desktop design is not: the screen is small, the input device is a fingertip, the network is unreliable, the battery is finite, and the user is frequently distracted and in motion. Good mobile UI design is largely the discipline of working within those constraints.

### 1. Screen size and device fragmentation

- There is very little space, so information must be prioritised ruthlessly - progressive disclosure, showing the primary action first and hiding secondary detail behind navigation.
- Android spans thousands of screen sizes, aspect ratios and pixel densities; iOS adds notches, dynamic islands and home indicators. Layouts must be responsive, using density-independent units - dp and sp on Android, points on iOS - constraint-based layout, and safe-area insets rather than fixed pixel positions.
- Foldables, tablets and split-screen multitasking mean the window can change size while the application is running.

### 2. Touch as the input method

- A fingertip is imprecise. The minimum touch target is 48 x 48 dp on Android and 44 x 44 pt on iOS, with adequate spacing to prevent mis-taps.
- Thumb-zone ergonomics: on a large phone held in one hand the bottom centre is easy to reach and the top corners are not. Primary actions belong in the lower reachable area, which is why bottom navigation bars replaced top tabs.
- No hover state exists, so affordances must be visible without one, and tooltips cannot be relied on.
- Gestures - swipe, pinch, long press, drag - are efficient but invisible. They need discoverable alternatives or onboarding, and they can conflict with system gestures such as edge-swipe back.

### 3. Platform conventions

- Android follows Material Design and iOS follows the Human Interface Guidelines. They differ in navigation patterns, a system back button versus a leading back control; in typography; in elevation and shadow; in control shapes; in date pickers and share sheets; and in where the tab bar sits.
- A cross-platform application must decide between one identical interface, which is cheaper and consistent with the brand, and per-platform adaptation, which feels correct to each user. Users notice when an application behaves like a foreigner on their device.

### 4. Performance, responsiveness and battery

- The interface must stay responsive at 60 frames per second, which allows roughly 16 ms per frame. Heavy work must move off the main or UI thread.
- Long lists must recycle views - RecyclerView, UICollectionView, lazy lists - rather than instantiate everything.
- Images must be sized and cached; loading a full-resolution photograph into a thumbnail is a common cause of jank and memory pressure.
- Animation should be limited to transform and opacity, which the compositor can handle, and must respect the reduce-motion accessibility setting.

- Background work, GPS polling and frequent network calls drain the battery, and an application that does so will be killed by the OS or uninstalled by the user.

## **5. Network conditions and offline behaviour**

- Connectivity is intermittent. An offline-first design keeps a local store as the source of truth and synchronises when a connection returns.
- Every asynchronous operation needs four designed states - loading, success, empty and error - with a retry path. Skeleton screens read as faster than spinners.
- Optimistic updates make the interface feel instant but require a rollback path when the request fails.

## **6. Accessibility**

- Screen readers, TalkBack and VoiceOver, need meaningful labels on every control and a sensible focus order; decorative images must be hidden from them.
- Text must scale with the system font-size setting without clipping, which rules out fixed-height text containers.
- Contrast must meet WCAG AA, 4.5:1 for body text, and colour must never be the only carrier of meaning - roughly eight per cent of men have a colour-vision deficiency.
- Support both light and dark themes, since dark mode is a system-level user preference.

## **7. Context of use**

- Mobile users are often interrupted, one-handed, outdoors in glare, or moving. Tasks must be short, resumable and forgiving, and state must survive the application being backgrounded or the process being killed.
- Forms are painful on a phone: minimise the number of fields, choose the right keyboard type, enable autofill, validate inline rather than on submit, and never ask for information that can be inferred.

## **8. Internationalisation**

- Translated strings can be thirty to forty per cent longer than English, so layouts must stretch. Right-to-left languages require mirrored layouts. Dates, numbers, currencies and name order are all locale dependent.

## **9. Permissions, privacy and trust**

- Request permissions in context and explain why, rather than demanding everything at first launch. Both platforms now penalise applications that over-ask, and users deny by reflex.

## **10. Consistency, feedback and testing**

- Use a design system - tokens for colour, type scale, spacing and elevation - so that the application is internally consistent and can be re-themed.
- Every action needs immediate feedback: pressed states, haptics, progress indication. An interface that appears to do nothing will be tapped again.
- Destructive actions need confirmation or an undo, and undo is usually the better pattern.
- Test on real low-end devices, not only on an emulator or a flagship. Validate with usability testing, and measure with analytics and crash reporting rather than relying on opinion.

**The underlying principle**

Every one of these considerations is a version of the same rule: reduce the user's cognitive and physical effort. Mobile design succeeds when the interface makes the next action obvious, reachable and reversible.

